

C# 프로그래밍

동명대학교 게임학부 · 2026년 1학기

중간고사 요약

C# 개념, 문법, 객체지향

Lecture 1 - 7 정리

강영민

동명대학교 게임학부

Contents

1 강의 전체 흐름	2
2 C# 기본 문법 (Lec 1-3)	2
2.1 프로그램 구조	2
2.2 주석	2
2.3 변수와 자료형	3
2.4 연산자	3
2.5 조건문	3
2.6 반복문	4
2.7 배열	4
3 클래스 기초 (Lec 4)	6
3.1 클래스란?	6
3.2 클래스의 구조	6
3.3 생성자 (Constructor)	7
3.4 프로퍼티 (Property)	7
3.5 컬렉션에 객체 저장하기	8
4 상속과 다형성 (Lec 5-6)	9
4.1 상속 (Inheritance)	9
4.2 다형성 (Polymorphism)	9
4.3 절차적 프로그래밍 vs OOP 비교	10
5 접근 제한자	11
6 Windows Forms 그래픽 (Lec 7)	12
6.1 프로젝트 설정 (.csproj)	12
6.2 Form, Graphics, OnPaint	12
6.3 게임 루프: Timer와 Invalidate	12
6.4 키보드 입력	13
6.5 클래스 기반 게임 구조	14
7 종합 정리	15

1. 강의 전체 흐름

강의	주제	핵심 개념	프로젝트
1	환경 설정 및 개요	IDE, .NET SDK 설치	—
2	C# 구문	프로그램 기본 구조	—
3	C# 기초 문법	변수, 연산자, 반복문, 배열	Unity 미니 프로젝트
4	클래스 기초	클래스, 생성자, 프로퍼티	Game of Life
5	클래스 활용	상속, 다형성, 클래스 헵	콘솔 게임 3종
6	절차적 vs OOP	패러다임 비교	같은 게임, 두 방식
7	그래픽과 슈팅 게임	Windows Forms, 게임 루프	10단계 슈팅 게임

2. C# 기본 문법 (Lec 1-3)

2.1. 프로그램 구조

C# 프로그램은 다음과 같은 구조로 작성된다:

```

1 using System; // library reference
2
3 class Program
4 {
5     static void Main() // entry point
6     {
7         Console.WriteLine("Hello, C#!");
8     }
9 }
```

핵심 포인트

- `using` — 사용할 라이브러리(네임스페이스)를 선언
- `class` — C#의 모든 코드는 클래스 안에 작성해야 한다
- `static void Main()` — 프로그램 실행이 시작되는 **진입점**
- 모든 문장은 세미콜론(;)으로 끝난다
- 코드 블록은 중괄호({ })로 감싼다

2.2. 주석

```

// single-line comment

/* multi-line
   comment */
```

2.3. 변수와 자료형

변수는 데이터를 저장하는 “이름 붙은 그릇”이다. 선언할 때 반드시 타입을 지정해야 한다.

타입	설명	예시	기본값
int	정수	int age = 20;	0
float	실수 (작은 범위)	float f = 3.14f;	0.0f
double	실수 (큰 범위)	double d = 3.14;	0.0
char	문자 하나	char c = 'A';	'\0'
string	문자열	string s = "hello";	null
bool	참/거짓	bool b = true;	false

형 변환 (Type Casting)

```
// implicit: small -> large (automatic)
int a = 10;
double b = a;           // int -> double, OK

// explicit: large -> small (cast required)
double x = 3.99;
int y = (int)x;         // 3 (decimal truncated)

// string conversion
string s = "42";
int n = int.Parse(s);   // string -> int
```

2.4. 연산자

분류	연산자	예시
산술	+ - * / %	10 / 3 → 3
비교	== != < > <= >=	5 == 5 → true
논리	&& !	true && false → false
대입	= += -= *= /=	x += 3은 x = x + 3
증감	++ --	i++은 i = i + 1

정수 나눗셈 vs 실수 나눗셈

```
int a = 10 / 3;           // 3 (both int -> integer division)
double b = 10.0 / 3;      // 3.333... (double -> real division)
```

2.5. 조건문

```
1 // if-else
2 if (score >= 90)
3     Console.WriteLine("A");
4 else if (score >= 80)
```

```

5     Console.WriteLine("B");
6 else
7     Console.WriteLine("C");
8
9 // switch
10 switch (grade)
11 {
12     case "A": Console.WriteLine("Excellent"); break;
13     case "B": Console.WriteLine("Good");      break;
14     default:  Console.WriteLine("Try again"); break;
15 }

```

2.6. 반복문

```

1 // for: repetition count is known
2 for (int i = 0; i < 10; i++)
3 {
4     Console.WriteLine(i);
5 }
6
7 // while: termination condition is important
8 int n = 0;
9 while (n < 10)
10 {
11     Console.WriteLine(n);
12     n++;
13 }

```

break와 continue

- break — 반복문을 즉시 종료
- continue — 현재 반복을 건너뛰고 다음 반복으로

```

for (int i = 0; i < 10; i++)
{
    if (i == 3) continue; // 3 skip
    if (i == 7) break;    // 7 stop
    Console.WriteLine(i); // output: 0, 1, 2, 4, 5, 6
}

```

2.7. 배열

배열은 같은 타입의 값을 여러 개 저장하는 자료구조이다.

```

1 // declaration and initialization
2 int[] scores = { 90, 85, 70, 95, 60 };
3
4 // access by index (starts from 0)

```

```
5 Console.WriteLine(scores[0]);    // 90
6 Console.WriteLine(scores[4]);    // 60
7
8 // iterate with loop
9 for (int i = 0; i < scores.Length; i++)
10 {
11     Console.WriteLine(scores[i]);
12 }
```

배열의 특징

- 인덱스는 0부터 시작: 첫 번째 원소는 arr[0]
- arr.Length로 원소 개수를 알 수 있다
- 크기는 생성 시 고정: 원소를 추가하거나 삭제할 수 없다

3. 클래스 기초 (Lec 4)

3.1. 클래스란?

클래스는 데이터와 기능을 하나로 묶는 설계도이다. 객체는 이 설계도로 만든 실체(인스턴스)이다.

비유

- 클래스 = 자동차 설계도 (도면)
- 객체 = 설계도로 만든 실제 자동차 (인스턴스)
- 하나의 설계도로 여러 대의 자동차(객체)를 만들 수 있다

3.2. 클래스의 구조

```
1 class Player
2 {
3     // Field: object's data
4     public string Name;
5     public int Hp;
6     public int Attack;
7
8     // Constructor: auto-called on creation
9     public Player(string name, int hp, int attack)
10    {
11        Name = name;
12        Hp = hp;
13        Attack = attack;
14    }
15
16    // Method: object's behavior
17    public void ShowInfo()
18    {
19        Console.WriteLine(Name + " HP:" + Hp);
20    }
21 }
```

```
1 // create objects
2 Player p1 = new Player("Warrior", 100, 30);
3 Player p2 = new Player("Mage", 70, 50);
4
5 p1.ShowInfo(); // Warrior HP:100
6 p2.ShowInfo(); // Mage HP:70
```

클래스의 세 가지 구성 요소

구성 요소	역할	예시
필드 (Field)	객체가 저장하는 데이터	int Hp;
생성자 (Constructor)	객체 생성 시 초기화	Player(...)
메서드 (Method)	객체가 수행하는 기능	void ShowInfo()

3.3. 생성자 (Constructor)

생성자는 new로 객체를 만들 때 **자동으로 호출되는** 특수 메서드이다.

```

1 class Enemy
2 {
3     public int X;
4     public int Y;
5     public int Speed;
6
7     // constructor: initialize fields
8     public Enemy(int x, int y, int speed)
9     {
10         X = x;
11         Y = y;
12         Speed = speed;
13     }
14 }
15
16 // new -> constructor auto-invoked
17 Enemy e = new Enemy(100, 200, 3);
18 // e.X = 100, e.Y = 200, e.Speed = 3

```

생성자 규칙

- 이름이 클래스 이름과 동일해야 한다
- 반환 타입이 없다 (void도 쓰지 않는다)
- 자동 호출된다 — 직접 이름으로 호출하지 않는다
- 생성자를 작성하지 않으면 C#이 매개변수 없는 기본 생성자를 제공한다

3.4. 프로퍼티 (Property)

프로퍼티는 get/set을 통해 필드에 안전하게 접근하는 C# 기능이다.

```

1 class Player
2 {
3     private int hp; // field: hidden from outside
4
5     public int Hp // property: controlled access
6     {

```



```

7         get { return hp; }
8         set
9         {
10             if (value < 0) hp = 0;    // prevent negative HP
11             else hp = value;
12         }
13     }
14 }

// auto-implemented property (no special logic needed)
public string Name { get; set; }

```

3.5. 컬렉션에 객체 저장하기

여러 객체를 배열이나 List에 저장하고 반복문으로 처리할 수 있다.

```

1 // object array
2 Enemy[] enemies = new Enemy[3];
3 enemies[0] = new Enemy(100, 0, 2);
4 enemies[1] = new Enemy(300, 0, 3);
5 enemies[2] = new Enemy(500, 0, 1);
6
7 for (int i = 0; i < enemies.Length; i++)
8 {
9     enemies[i].Update();
10 }

```

```

1 // List: dynamic add/remove
2 List<Bullet> bullets = new List<Bullet>();
3 bullets.Add(new Bullet(100, 500)); // add
4 bullets.RemoveAt(0);               // remove

```

배열 vs List

항목	배열 (T[])	List<T>
크기	생성 시 고정	동적으로 변경 가능
추가/삭제	불가능	Add(), RemoveAt()
개수 확인	.Length	.Count
사용 예	고정 수 (플레이어)	가변 수 (총알, 적)

4. 상속과 다형성 (Lec 5-6)

4.1. 상속 (Inheritance)

상속이란 자식 클래스가 부모 클래스의 필드와 메서드를 물려받는 것이다.

```

1  // parent class
2  class GameNumber
3  {
4      protected int number;
5
6      public GameNumber(int num)
7      {
8          number = num;
9      }
10
11     public virtual bool Check(int guess)
12     {
13         return guess == number;
14     }
15 }
16
17 // child class: inherits and extends
18 class MyNumber : GameNumber
19 {
20     public MyNumber(int num) : base(num) { }
21
22     public override bool Check(int guess)
23     {
24         if (guess > number)
25             Console.WriteLine("Too high!");
26         else if (guess < number)
27             Console.WriteLine("Too low!");
28         return guess == number;
29     }
30 }

```

상속 키워드 정리

키워드	의미
class Child : Parent	Child가 Parent를 상속
base(num)	부모의 생성자를 호출
virtual	“자식이 재정의할 수 있는 메서드”
override	“부모의 메서드를 재정의한다”
protected	자신과 자식 클래스만 접근 가능

4.2. 다형성 (Polymorphism)

다형성이란 “같은 이름, 다른 동작”이다. 부모 타입 변수에 자식 객체를 담으면, 자식이 재정의한 메서드가 호출된다.

```

1 GameNumber g = new MyNumber(42);
2 g.Check(50); // MyNumber's Check is called
3              // output: "Too high!"

```

다형성이 중요한 이유

서로 다른 동작을 하는 객체들이 같은 인터페이스(메서드 이름)를 공유하면, 구체적인 타입을 신경 쓰지 않고 일괄 처리할 수 있다. 게임에서 다양한 종류의 적이나 아이템을 관리할 때 널리 쓰이는 원리이다.

4.3. 절차적 프로그래밍 vs OOP 비교

Lec 6에서는 같은 게임을 두 가지 방식으로 구현하여 비교하였다:

두 접근법 비교

항목	절차적	OOP
데이터 관리	모든 변수가 한곳에 모여 있음	각 클래스가 자기 데이터를 관리
코드 수정	전체 흐름을 추적해야 함	해당 클래스만 수정
재사용	복사-붙여넣기 필요	클래스를 가져다 사용
가독성	긴 순차 코드를 읽어야 함	클래스 이름으로 의도 파악

클래스를 쓰면 좋은 세 가지 이유

1. **역할 분리** — 각 클래스가 하나의 책임만 담당
2. **수정 용이** — 기능 변경 시 해당 클래스만 편집
3. **재사용 가능** — 한 번 만든 클래스를 다른 프로젝트에서 사용

5. 접근 제한자

접근 제한자는 “누가 이 멤버에 접근할 수 있는가”를 제어한다.

제한자	접근 범위	용도
public	어디서든 접근 가능	외부 인터페이스 (메서드)
private	같은 클래스 안에서만	내부 데이터 (필드)
protected	자신 + 자식 클래스	상속으로 공유할 데이터

```

1 class Player
2 {
3     private int hp = 100;           // Player only
4     protected int speed = 5;       // child classes too
5     public string Name = "P1";     // anyone
6
7     public void TakeDamage(int dmg)
8     {
9         hp -= dmg; // private field: internal access
10    }
11 }

```

기본 접근 수준

C#에서 접근 제한자를 생략하면:

- 클래스 멤버(필드, 메서드)는 `private`이 기본
- 클래스 자체는 `internal`이 기본 (같은 프로젝트 내 접근 가능)

override 시 접근 제한자 규칙

메서드를 override할 때, 부모보다 접근 범위를 좁힐 수 없다.

부모 선언	자식에서 사용 가능	불가능
<code>protected virtual</code>	<code>protected override</code> <code>public override</code>	<code>private override</code>

관례적으로 부모와 같은 접근 수준을 유지한다.

6. Windows Forms 그래픽 (Lec 7)

6.1. 프로젝트 설정 (.csproj)

Windows Forms를 사용하려면 프로젝트 파일에 다음 설정이 필요하다:

```

1 <Project Sdk="Microsoft.NET.Sdk">
2   <PropertyGroup>
3     <OutputType>WinExe</OutputType>
4     <TargetFramework>net10.0-windows</TargetFramework>
5     <UseWindowsForms>true</UseWindowsForms>
6   </PropertyGroup>
7 </Project>

```

UseWindowsForms가 없으면 Form, Graphics 등의 클래스를 찾을 수 없어 컴파일 에러가 발생한다. .cs 파일이 “레시피(코드)”라면, .csproj는 “재료 목록(어떤 라이브러리를 쓸 것인가)”이다.

6.2. Form, Graphics, OnPaint

```

1 class GameForm : Form
2 {
3     public GameForm()
4     {
5         Text = "My Game";
6         ClientSize = new Size(800, 600);
7         BackColor = Color.Black;
8         DoubleBuffered = true;
9     }
10
11     protected override void OnPaint(PaintEventArgs e)
12     {
13         Graphics g = e.Graphics;
14         g.FillRectangle(Brushes.White, 400, 300, 2, 2);
15         g.FillEllipse(Brushes.Cyan, 100, 100, 40, 40);
16         g.DrawLine(Pens.Yellow, 0, 0, 800, 600);
17     }
18 }

```

protected override의 의미

- override — 부모(Form)의 OnPaint를 우리 코드로 대체한다. 없으면 부모의 빈 OnPaint가 호출되어 화면에 아무것도 그려지지 않는다.
- protected — 자신과 자식 클래스에서 접근 가능. public도 가능하지만 private는 컴파일 에러 (부모보다 범위를 좁힐 수 없다).

6.3. 게임 루프: Timer와 Invalidate

```

1 // in the constructor:
2 timer = new Timer();

```

```

3 timer.Interval = 16;           // 16ms ~ 60fps
4 timer.Tick += OnUpdate;        // register event handler
5 timer.Start();
6
7 // called every 16ms:
8 void OnUpdate(object sender, EventArgs e)
9 {
10     x += dx;
11     y += dy;
12     Invalidate(); // request repaint -> OnPaint
13 }

```

게임 루프 흐름

Timer Tick → OnUpdate (상태 갱신) → Invalidate() → OnPaint (화면 다시 그림) → 반복

이 과정이 초당 약 60번 반복되면서 “움직이는” 것처럼 보인다.

timer.Tick += OnUpdate;의 의미

코드	의미
timer.Tick	Timer가 Interval마다 발생시키는 이벤트
+=	이벤트에 메서드를 등록(구독)
OnUpdate	이벤트 발생 시 실행될 메서드

괄호 없이 OnUpdate만 쓰는 이유: 지금 호출하는 것이 아니라 메서드의 **참조**를 넘기기 때문이다. OnUpdate()은 즉시 호출, OnUpdate은 참조 전달이다.

6.4. 키보드 입력

```

1 HashSet<Keys> keys = new HashSet<Keys>();
2
3 protected override void OnKeyDown(KeyEventArgs e)
4 {
5     keys.Add(e.KeyCode);
6 }
7
8 protected override void OnKeyUp(KeyEventArgs e)
9 {
10    keys.Remove(e.KeyCode);
11 }
12
13 void OnUpdate(object sender, EventArgs e)
14 {
15     if (keys.Contains(Keys.Left)) x -= speed;
16     if (keys.Contains(Keys.Right)) x += speed;
17     Invalidate();
18 }

```

왜 HashSet을 쓰는가?

키를 누르고 있으면 OnKeyDown이 반복 호출된다. HashSet은 중복을 무시하므로 키 하나당 항목 하나만 유지되어, “현재 눌린 키 목록”을 정확하게 관리할 수 있다.

6.5. 클래스 기반 게임 구조

기능이 늘어나면 코드를 개별 클래스로 분리한다:

클래스	담당	주요 메서드
Program	진입점 (Main)	Application.Run()
GameForm	게임 루프, 이벤트 처리	OnUpdate, OnPaint
Player	플레이어 위치, 입력	Update(), Draw()
Bullet	총알 이동, 경계 확인	Update(), Draw()
Enemy	적 이동, 충돌 판정	Update(), HitBy()
Hud	점수, 생명 표시	Draw()

각 클래스가 자기 데이터와 행동을 관리하고, GameForm이 모든 객체를 생성하고 조율한다.

7. 종합 정리

핵심 개념 체크리스트

1. 변수와 타입 — int, double, string, bool의 사용법
2. 제어 흐름 — if/else, switch, for, while, break/continue
3. 배열과 컬렉션 — int[], List<T>, HashSet<T>
4. 클래스 — 데이터(필드) + 기능(메서드)의 설계도
5. 생성자 — new 시 자동 호출, 객체를 초기화
6. 상속 — class Child : Parent로 재사용 및 확장
7. 다형성 — virtual/override: 같은 이름, 다른 동작
8. 접근 제한자 — public (모두), private (자신만), protected (자신+자식)
9. 이벤트 — timer.Tick += OnUpdate: 메서드를 이벤트에 등록
10. 게임 루프 — Timer → Update → Invalidate → OnPaint

OOP 핵심 원리: 역할 분리

“각 클래스는 **하나의 책임**만 담당한다.”

- Player는 “자기 자신을 이동하고 그리는 것”만 안다
- Bullet은 “날아가고 경계를 확인하는 것”만 안다
- GameForm은 “전체 게임을 조율하는 것”을 담당한다

이렇게 하면 하나의 기능을 수정할 때 **해당 클래스만 변경**하면 된다. 이것이 클래스를 사용하는 핵심 이유이다.